**TIP** Wherever possible, I recommend using a third-party identity provider. Well-respected identity providers have many experts working solely on securing your customers' details, proactively preventing attacks and ensuring that the data is safe. By leaving this tricky job to the experts, you're free to focus on the core business of your app, whatever that may be.

Another common option is to build your *own* identity provider. This may sound like a lot of work (and it is!), but thanks to excellent libraries like OpenIddict (https:// github.com/openiddict) and Duende's IdentityServer (https://duendesoftware.com), it's perfectly possible to write your own identity provider to serve bearer tokens that can be consumed by your apps and APIs.

**WARNING** You should consider carefully whether the effort and risks associated with creating your own identity provider are worthwhile. Bugs are a fact of life, and a bug in your identity provider could easily result in a security vulnerability. Nevertheless, if you have specific identity requirements, creating your own identity provider may be a reasonable or necessary option.

An aspect often overlooked by people getting started with OpenIddict and IdentityServer is that they aren't prefabricated solutions. They consist of a set of services and middleware that you add to a standard ASP.NET Core app, providing an implementation of relevant identity standards, according to the specification. You, as a developer, still need to write the profile management code that knows how to create a new user (normally in a database), load a user's details, validate their password, and manage their associated claims. On top of that, you need to provide all the UI code for the user to log in, manage their passwords, and configure multi-factor authentication (MFA). It's not for the faint of heart!